

PRO2 Probeklausur

Klausurtraining mit vollständiger Musterlösung - orientiert an PRO_Klausurinfo.md

Rahmenbedingungen: 90 Minuten, 100 Punkte, schriftlich auf Papier. Schwerpunkt liegt auf kurzen Coding- und Verständnisaufgaben. Backtracking und Heapsort sind bewusst nicht enthalten; ATG/LL(1) ist enthalten.

Hinweis: Die Aufgaben sind bewusst etwas dichter als eine echte Klausur, damit möglichst viele klausurrelevante Schwerpunkte trainiert werden.

Bearbeitungshinweise: Begründen Sie kurze Antworten knapp, aber präzise. Bei unklaren Details dürfen Sie Annahmen treffen, müssen diese aber notieren. Java-Code muss nicht importfrei lauffähig sein, soll aber logisch korrekt sein.

Aufgabenübersicht

Nr.	Thema	Punkte	Richtzeit
1	Java-Grundlagen, Felder, Stringvergleich	14	12 min
2	Klassen und Objekte: Rational	16	16 min
3	Vererbung, Polymorphie, dynamische Bindung	14	12 min
4	Ausnahmebehandlung und Sichtbarkeit	10	8 min
5	ATG/Grammatiken und LL(1)	18	18 min
6	Hashing-basierte Suche	12	10 min
7	Heap/Priority Queue	8	7 min
8	Graphen und objektorientierter Entwurf	8	7 min
	Summe	100	90 min

Teil A - Aufgabenblatt

Klausurmodus: Lösen Sie diesen Teil zunächst ohne Lösungsteil. Die Lösung folgt ab Teil B.

Aufgabe 1: Java-Grundlagen, Felder und Stringvergleich (14 Punkte)

(a) Zeichnen Sie die Datenstrukturen nach Ausführung der folgenden Anweisungen. Geben Sie insbesondere an, welche Variablen/Arraypositionen auf dasselbe Objekt zeigen und welche Positionen den Defaultwert besitzen. (5 P)

```
String s = "Hi";
String t = new String("Hi");
String[] a = new String[4];
String[] b = a;
a[0] = s;
a[2] = t;
b[1] = a[0] + "!";
b[3] = null;
```

(b) Bestimmen Sie jeweils Ergebnis oder Fehler. Begründen Sie kurz. (4 P)

Ausdruck	Ergebnis/Fehler
<code>a == b</code>	
<code>a[0] == s</code>	
<code>a[0] == t</code>	
<code>a[0].equals(t)</code>	
<code>a[3] == null</code>	
<code>a[3].equals(s)</code>	

(c) Implementieren Sie die Klassenmethode `countEquals`. Sie soll zählen, wie viele nicht-null-Elemente im Feld `text` inhaltlich gleich `value` sind. Falls `text` oder `value` null ist, soll 0 zurückgegeben werden. (5 P)

```
public static int countEquals(String[] text, String value) {
    // ...
}
```

Aufgabe 2: Klassen und Objekte: Rational (16 Punkte)

Implementieren Sie die Klasse `Rational` für rationale Zahlen. Anforderungen:

- Datenkomponenten sind `private` und unveränderlich: `num` und `denom`.
- Standardkonstruktor erzeugt 0/1 und ruft den allgemeinen Konstruktor auf.
- Allgemeiner Konstruktor validiert `denom != 0`, normalisiert negatives Vorzeichen in den Zähler und kürzt den Bruch.
- Objektmethode `multiply(Rational r)` liefert ein neues `Rational`-Objekt und verändert `this` nicht.
- `toString`, `equals(Object)` und `hashCode` werden sinnvoll überschrieben.

```
public final class Rational {
    // Datenkomponenten, Konstruktoren und Methoden
}
```

Aufgabe 3: Vererbung, Polymorphie und dynamische Bindung (14 Punkte)

(a) Welche Ausgabe entsteht? Schreiben Sie die Ausgabe zeilenweise auf. (8 P)

```
class A {
    void f() { IO.print("A"); }
    void g() { f(); IO.print("g"); }
    void h(A x) { IO.print("HA"); }
}
```

```

class B extends A {
    void f() { IO.print("B"); }
    void h(B x) { IO.print("HB"); }
    void k() { super.f(); f(); }
}

A a = new A();
A x = new B();
B b = new B();

a.g(); IO.println();
x.g(); IO.println();
b.k(); IO.println();
x.h(b); IO.println();
b.h(b); IO.println();

```

(b) Erklären Sie kurz den Unterschied zwischen Überschreiben und Überladen. Beziehen Sie sich dabei auf f und h. (4 P)

(c) Warum ruft f() innerhalb von A.g bei x.g() trotzdem B.f auf? (2 P)

Aufgabe 4: Ausnahmebehandlung und Sichtbarkeit (10 Punkte)

(a) Geben Sie für die drei Aufrufe Ausgabe und Ergebnis/Ausnahme an. (6 P)

```

public static int parsePositive(String s) throws IOException {
    try {
        int n = Integer.parseInt(s);
        if (n < 0) {
            throw new IllegalArgumentException("negative");
        }
        return n;
    } catch (NumberFormatException e) {
        IO.print("N");
        throw new IOException("not a number");
    } finally {
        IO.print("F");
    }
}

parsePositive("7");
parsePositive("-2");
parsePositive("x");

```

(b) Welche der auftretenden Ausnahmen müssen deklariert oder behandelt werden? Warum? (2 P)

(c) Eine Klasse C ist ohne public im Paket util deklariert. Darf eine Klasse app.App aus einem anderen Paket C direkt verwenden? Begründen Sie. (2 P)

Aufgabe 5: ATG, Terminalklassen und LL(1) (18 Punkte)

Eine einfache Kommandosprache soll Bewegungen und Einstellungen beschreiben. Beispiele:

```

MOVE x 10 cm; WAIT 50 ms; SET speed = 3.
MOVE y 2 cm.

```

Bedeutung: Ein Programm besteht aus mindestens einem Kommando, Kommandos werden mit Semikolon getrennt, das gesamte Programm endet mit einem Punkt. Ein Kommando ist entweder MOVE ident number cm, WAIT number ms oder SET ident = number.

(a) Geben Sie die Terminalklassen ident und number sprachlich oder mit Regex an. Nennen Sie außerdem die festen Terminalsymbole. (4 P)

(b) Formulieren Sie eine EBNF-Grammatik für die Sprache. (7 P)

(c) Begründen Sie kurz, warum die Alternativen für Command LL(1)-konform sind. (3 P)

(d) Die folgende Regel ist nicht LL(1)-konform. Schreiben Sie sie LL(1)-konform um und begründen Sie kurz. (4 P)

```

UnitValue = number | number "cm" | number "m" | ident.

```

Aufgabe 6: Hashing-basierte Suche (12 Punkte)

Gegeben ist $m = 7$ und $H(k) = k \bmod 7$. Fügen Sie die Schlüssel in dieser Reihenfolge ein: 10, 17, 24, 3, 11.

- (a) Zeichnen Sie die Hashtabelle bei Kollisionsbehandlung durch Verkettung. Listen in Einfügereihenfolge. (4 P)
- (b) Zeichnen Sie die Hashtabelle bei offener Adressierung mit linearem Sondieren. (4 P)
- (c) Wie viele Tabellenpositionen werden beim Suchen von 24 mit linearem Sondieren geprüft? (2 P)
- (d) Nennen Sie zwei Eigenschaften einer guten Hashfunktion und erklären Sie, wann man durchschnittlich $O(1)$ erwarten darf. (2 P)

Aufgabe 7: Heap/Priority Queue (8 Punkte)

Gegeben ist ein Max-Heap in Feldform. Position 0 ist unbenutzt.

Index:	0	1	2	3	4	5	6	7	8	9
heap:	-	21	14	18	8	10	12	7	3	5

- (a) Fügen Sie 20 ein und geben Sie das Feld nach jedem notwendigen Tausch oder mindestens den Endzustand an. (4 P)
- (b) Entfernen Sie danach das Maximum und geben Sie den Endzustand an. (3 P)
- (c) Warum ist die Laufzeit von insert und removeMax $O(\log n)$? (1 P)

Aufgabe 8: Graphen und objektorientierter Entwurf (8 Punkte)

(a) Für den gerichteten Graphen $V = \{1,2,3,4,5\}$, $E = \{(1,2), (1,3), (2,4), (3,4), (4,2), (4,5)\}$: Geben Sie die Adjazenzlisten, InDegree(4), OutDegree(4), einen Weg von 1 nach 5 und einen Zyklus an. (4 P)

(b) Implementieren Sie für eine Adjazenzlistenrepräsentation `int[][] adj` eine Methode `hasEdge`. Es darf angenommen werden, dass `v` und `w` gültige Knotenindizes sind. (2 P)

```
public boolean hasEdge(int v, int w) {
    // adj[v] enthält alle direkten Nachfolger von v
}
```

(c) Eine Datei und ein Verzeichnis sollen beide `size()` und `print(indent)` unterstützen. Ein Verzeichnis kann Dateien und weitere Verzeichnisse enthalten. Nennen Sie die zentralen Klassen/Interfaces eines Composite-Entwurfs. (2 P)

Teil B - Musterlösung

Bewertungshinweis: Die Lösungen zeigen eine mögliche Musterlösung. Andere korrekte Java-Varianten sind gleichwertig, wenn Schnittstelle, Semantik und Begründung stimmen.

Lösung 1: Java-Grundlagen, Felder und Stringvergleich

(a) s referenziert das String-Objekt "Hi". t referenziert ein anderes String-Objekt mit gleichem Inhalt. a und b referenzieren dasselbe Arrayobjekt der Länge 4. Nach new String[4] sind alle vier Positionen zunächst null.

Index	a[index] / b[index]	Bedeutung
0	s	referenziert dasselbe Objekt wie s, Inhalt "Hi"
1	neues String-Objekt	Ergebnis von a[0] + "!", Inhalt "Hi!"
2	t	referenziert dasselbe Objekt wie t, Inhalt "Hi"
3	null	explizit null, entspricht auch dem Defaultwert für Referenzfelder

(b)

Ausdruck	Ergebnis	Begründung
a == b	true	a und b zeigen auf dasselbe Arrayobjekt.
a[0] == s	true	a[0] wurde direkt auf s gesetzt.
a[0] == t	false	s und t sind verschiedene Objekte, auch wenn der Inhalt gleich ist.
a[0].equals(t)	true	equals vergleicht bei String den Inhalt.
a[3] == null	true	a[3] ist null.
a[3].equals(s)	NullPointerException	Auf null kann keine Methode aufgerufen werden.

(c)

```
public static int countEquals(String[] text, String value) {
    if (text == null || value == null) {
        return 0;
    }
    int count = 0;
    for (int i = 0; i < text.length; i++) {
        if (value.equals(text[i])) { // robust: text[i] darf null sein
            count++;
        }
    }
    return count;
}
```

Lösung 2: Klassen und Objekte: Rational

```
public final class Rational {
    private final int num;
    private final int denom;

    public Rational() {
        this(0, 1);
    }
}
```

```

public Rational(int num, int denom) {
    if (denom == 0) {
        throw new IllegalArgumentException("denom == 0");
    }
    if (denom < 0) {
        num = -num;
        denom = -denom;
    }
    int g = gcd(Math.abs(num), denom);
    this.num = num / g;
    this.denom = denom / g;
}

private static int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a == 0 ? 1 : a;
}

public Rational multiply(Rational r) {
    if (r == null) {
        throw new IllegalArgumentException("r == null");
    }
    return new Rational(this.num * r.num, this.denom * r.denom);
}

@Override
public String toString() {
    if (denom == 1) {
        return "" + num;
    }
    return num + "/" + denom;
}

@Override
public boolean equals(Object obj) {
    if (!(obj instanceof Rational)) {
        return false;
    }
    Rational r = (Rational) obj;
    return this.num == r.num && this.denom == r.denom;
}

@Override
public int hashCode() {
    return 31 * num + denom;
}
}

```

Wichtig: equals muss Object als Parametertyp haben, sonst wird Object.equals nicht überschrieben, sondern nur überladen. Der Zugriff r.num ist erlaubt, obwohl num private ist, weil private klassenspezifisch und nicht objektspezifisch gilt.

Lösung 3: Vererbung, Polymorphie und dynamische Bindung

(a) Ausgabe:

```

Ag
Bg
AB
HA
HB

```

Begründung:

- a.g(): Objekt ist A. A.g ruft A.f auf und gibt danach g aus -> Ag.
- x.g(): statischer Typ A, dynamisches Objekt B. A.g wird verwendet, aber der Aufruf f() wird dynamisch gebunden -> B.f, daher Bg.
- b.k(): super.f() erzwingt A.f; danach f() dynamisch auf B.f -> AB.
- x.h(b): Beim Überladen wird die Signatur zur Übersetzungszeit nach statischem Typ von x gewählt. Sichtbar ist A.h(A). B.h(B) überschreibt A.h(A) nicht -> HA.

- $b.h(b)$: Beim statischen Typ B ist $B.h(B)$ die spezifischste überladene Methode $\rightarrow HB$.

(b) Überschreiben: gleiche Signatur in Unterklasse, dynamische Bindung zur Laufzeit, z. B. $B.f$ überschreibt $A.f$.

Überladen: gleicher Methodenname, aber andere Parameterliste, Auswahl nach statischen Typen zur Übersetzungszeit, z. B. $h(A)$ und $h(B)$.

(c) Methodenaufrufe auf Objekten werden in Java dynamisch gebunden. Auch der unqualifizierte Aufruf $f()$ in $A.g$ bedeutet sinngemäß $this.f()$; $this$ referenziert bei $x.g()$ ein B -Objekt.

Lösung 4: Ausnahmebehandlung und Sichtbarkeit

Aufruf	Ausgabe	Ergebnis
<code>parsePositive("7")</code>	F	return 7
<code>parsePositive("-2")</code>	F	<code>IllegalArgumentException</code> wird geworfen
<code>parsePositive("x")</code>	NF	<code>NumberFormatException</code> wird gefangen, danach <code>IOException</code> geworfen

finally wird in allen drei Fällen ausgeführt: bei normalem return, bei `RuntimeException` und beim erneuten Werfen einer checked Exception.

(b) `IOException` ist eine checked Exception und muss deshalb mit `throws` deklariert oder mit `catch` behandelt werden. `NumberFormatException` und `IllegalArgumentException` sind `RuntimeExceptions` und müssen nicht deklariert werden.

(c) Nein. Eine Top-Level-Klasse ohne `public` ist nur paketweit sichtbar. `app.App` liegt in einem anderen Paket und darf `util.C` daher nicht direkt verwenden.

Lösung 5: ATG, Terminalklassen und LL(1)

(a) Eine mögliche lexikalische Festlegung:

- `ident`: Buchstabe, gefolgt von beliebig vielen Buchstaben/Ziffern/Unterstrichen, z. B. Regex `[A-Za-z][A-Za-z0-9_]*`.
- `number`: mindestens eine Ziffer, z. B. Regex `[0-9]+`.
- Feste Terminalsymbole: `MOVE`, `WAIT`, `SET`, `cm`, `ms`, `=`, `;`, `,`.

(b) Eine LL(1)-geeignete EBNF-Grammatik:

```

Program = Command { ";" Command } "." .
Command = Move | Wait | Set .
Move    = "MOVE" ident number "cm" .
Wait    = "WAIT" number "ms" .
Set     = "SET" ident "=" number .
ident   = letter { letter | digit | "_" } .
number  = digit { digit } .

```

(c) $FIRST(Move) = \{MOVE\}$, $FIRST(Wait) = \{WAIT\}$, $FIRST(Set) = \{SET\}$. Die FIRST-Mengen der Alternativen sind paarweise disjunkt; daher kann ein rekursiver Abstieg anhand eines Lookahead-Symbols die richtige Alternative wählen.

(d) Nicht LL(1), weil drei Alternativen mit `number` beginnen. LL(1)-konforme Umformung:

```

UnitValue = number [ "cm" | "m" ] | ident .

```

Jetzt sind die FIRST-Mengen der obersten Alternativen $\{number\}$ und $\{ident\}$ disjunkt. Nach `number` entscheidet das nächste Symbol optional über `cm`, `m` oder das Ende der Regel.

Lösung 6: Hashing-basierte Suche

Hashwerte: $H(10)=3$, $H(17)=3$, $H(24)=3$, $H(3)=3$, $H(11)=4$.

(a) Verkettung in Einfügereihenfolge:

Index	Liste
0	-
1	-
2	-
3	10 -> 17 -> 24 -> 3
4	11
5	-
6	-

(b) Offene Adressierung mit linearem Sondieren:

Index	0	1	2	3	4	5	6
Wert	11	-	-	10	17	24	3

(c) Suche nach 24 mit linearem Sondieren prüft Index 3, 4, 5; also 3 Tabellenpositionen.

(d) Eine gute Hashfunktion ist schnell berechenbar und verteilt typische Schlüssel möglichst gleichmäßig auf die Tabelle. Durchschnittlich $O(1)$ ist realistisch, wenn die Hashfunktion gut verteilt und der Füllgrad der Tabelle kontrolliert bleibt; bei vielen Kollisionen oder zu hoher Belegung verschlechtert sich die Suche.

Lösung 7: Heap/Priority Queue

(a) Einfügen von 20: 20 wird an Position 10 eingefügt. Es steigt über 10 und danach über 14, stoppt unter 21.

```
Start:      21 14 18  8 10 12  7  3  5
Einfügen:   21 14 18  8 10 12  7  3  5 20
Tausch 20/10: 21 14 18  8 20 12  7  3  5 10
Tausch 20/14: 21 20 18  8 14 12  7  3  5 10
```

(b) removeMax danach: 21 wird entfernt, 10 kommt an die Wurzel und sinkt über 20 und dann über 14.

```
Nach Entfernen der Wurzel: 10 20 18  8 14 12  7  3  5
Tausch 10/20:      20 10 18  8 14 12  7  3  5
Tausch 10/14:      20 14 18  8 10 12  7  3  5
```

(c) Der Heap ist ein kompletter Binärbaum. Seine Höhe ist logarithmisch in n . swim und sink laufen höchstens entlang eines Pfads von Blatt zu Wurzel bzw. Wurzel zu Blatt: $O(\log n)$.

Lösung 8: Graphen und objektorientierter Entwurf

(a) Adjazenzlisten:

Knoten	Nachfolger
1	2, 3
2	4
3	4
4	2, 5
5	-

InDegree(4)=2, weil Kanten von 2 und 3 nach 4 zeigen. OutDegree(4)=2, weil Kanten von 4 nach 2 und 5 ausgehen. Ein Weg von 1 nach 5 ist 1 -> 2 -> 4 -> 5 oder 1 -> 3 -> 4 -> 5. Ein Zyklus ist 2 -> 4 -> 2.

(b)

```
public boolean hasEdge(int v, int w) {
    for (int i = 0; i < adj[v].length; i++) {
        if (adj[v][i] == w) {
            return true;
        }
    }
}
```



```
    return false;
}
```

(c) Composite-Entwurf:

- Gemeinsamer Typ, z. B. interface Entry mit size() und print(String indent).
- Leaf: class File implements Entry; enthält z. B. name und size.
- Composite: class Directory implements Entry; enthält eine Sammlung von Entry-Objekten und delegiert size/print rekursiv an Kinder.

```
interface Entry {
    int size();
    void print(String indent);
}

class File implements Entry { /* Leaf */ }
class Directory implements Entry { /* Composite: enthält Entry[] oder List<Entry> */ }
```

Interne Vollständigkeits-Checkliste

Ziel: Diese Probeklausur deckt die in der Klausurinfo betonten Themen ab und vermeidet bewusst ausgeschlossene Themen.

- 90-Minuten-Struktur mit 100 Punkten und kurzen, papiergeeigneten Aufgaben.
- Coding-Schwerpunkt: Felder/Stringvergleich, Klassenimplementierung, OOP-Ausgabe, Exceptions, ATG, Hashing, Heap, Graphmethode, Composite-Skizze.
- ATG/LL(1) kommt vor; Interpreter/Parserimplementierung wird nicht verlangt.
- Backtracking und Heapsort kommen nicht vor.
- Generics werden nicht als Hauptthema geprüft; Composite darf mit Entry[] oder einer gegebenen Sammlung gedacht werden.
- Aufgaben sind auf Verständnis und sichere Klausurargumentation ausgelegt.